

AI LESSON PLAN GENERATOR

End-to-End System Documentation & Blueprint

TECHNICAL BLUEPRINT & DOCUMENTATION

An in-depth guide to architecture, security, failover mechanisms, and the future development roadmap.

SYSTEM OVERVIEW

- AI Orchestration: Gemini 1.5 Flash (Primary) + Rule-Based Templates (Fallback)
- Backend Stack: Node.js 20, Express, PostgreSQL, Zod Input Validation
- Frontend Stack: React 18, Vite 5, Tailwind CSS, Axios Client
- Security Posture: JWT Auth, bcryptjs, Helmet, Rate Limiter, Clock Skew Tolerance
- End-to-End SLA: Under 10 seconds for all generation requests

Prepared by: Google DeepMind Antigravity AI & Irfan Codes

Document Version 1.1.0 · Generated on 6/8/2026

1. Introduction & Executive Summary

The AI Lesson Plan Generator is a tailored preschool curriculum generation application built specifically for early childhood educators. Preschool teachers operate in high-energy, fast-paced environments and require structured, highly age-appropriate lesson plans without administrative delay. This application addresses that need by enabling teachers to select an age band (from 2 to 6 years), choose a curriculum theme (such as Animals, Colors, or Weather), select a date, and instantaneously generate a complete plan.

Every lesson plan output contains a defined learning objective, a step-by-step activity, a classroom rhyme, a printable worksheet idea, and a detailed materials list. Real-time reliability is guaranteed via a deterministic failover architecture. If the Google Gemini API throws an exception, times out, or has no key configured, the system immediately switches to pre-authored static fallback templates, ensuring that a teacher in a classroom is never left without a lesson design.

2. Architectural Blueprint

The system is designed as a decoupled monorepo structured via npm workspaces. This provides clear boundaries between the frontend client and backend API while keeping shared configuration in one place.

Frontend Client Layer

- React 18 & Vite 5: Offers an extremely fast Single Page Application (SPA) development workflow and optimized assets production build.
- Tailwind CSS 3: Drives the "Playful Professionalism" visual design system using clean borders, soft drop shadows, and friendly color codes.
- Axios Client: Intercepts outgoing requests to append authorization tokens and intercepts incoming responses to handle 401 logouts.
- React Router 6: Handles client-side routing, protected dashboards, and redirect flows.

Backend API Layer

- Node.js 20 & Express 4: Powers the REST API routes, middleware validation, and database querying.
- Zod: Enforces schema validation on both request query strings and bodies, returning standardized 400 Bad Request responses on validation failures.
- Pino: Provides structured JSON logging on both info and error paths to facilitate troubleshooting.
- pdfkit: Creates and streams A4 formatted documents directly from in-memory objects to HTTP responses without temporary files.

3. Database & Core Schema

The database layer uses PostgreSQL 16 (fully Supabase-compatible) and handles schema migrations via the node-pg-migrate framework.

User Account Table (users)

- id (UUID, Primary Key): Identifies the teacher account.
- name (Text): Teacher name displayed on dashboard.
- email (Text, Unique): Teacher email address used for login authentication.
- password_hash (Text): Securely stored password via bcryptjs (BCRYPT_COST = 12).
- created_at (TimestampTZ): Automatic registration timestamp.

Lesson Plan Storage Table (lesson_plans)

- id (UUID, Primary Key): Identifies the saved lesson.
- user_id (UUID, Foreign Key -> users.id): Establishes record ownership.
- age_group (Text): The targeted classroom age range (e.g. 3-4, 4-5).
- theme (Text, Indexed): The selected theme to enable fast search queries.
- lesson_date (Date): The scheduled date for the lesson.
- lesson_content (JSONB): Formatted structure containing objective, activity, rhyme, worksheet, and materials.
- source (Text): Identifies the generation path ("gemini" or "fallback").
- created_at (TimestampTZ, Indexed Desc): Saved timestamp for sorted listing.

4. Authentication & Security Posture

Authentication uses JWT tokens. Because the frontend utilizes Supabase Auth (email/password and Google OAuth), the backend is designed to handle both remote verification via Supabase and a robust local decode fallback.

The Token Verification Workflow

- The frontend LoginPage logs the user in via Supabase Auth and stores data.session.access_token.
- Axios request interceptors automatically call supabase.auth.getSession() before every request to retrieve a fresh access token, auto-refreshing it in the background if it is close to expiration.
- The backend requireAuth middleware extracts the Bearer token and attempts remote verification via getSupabase().auth.getUser(token).
- If the remote check fails or Supabase is offline, the backend falls back to local JWT decoding, extracting claims and checking the token expiration.
- To account for client-server time offsets, a 5-minute (300 seconds) clock-skew tolerance is enforced in the backend token validation check.
- If verification fails entirely, the backend returns a 401 Unauthorized response, prompting the frontend Axios response interceptor to clear local storage and redirect the user back to the login screen.

5. AI Orchestrator & Fallback Engine

The core generation endpoint, POST /api/lessons/generate, is controlled by an orchestrator designed to satisfy a strict 10-second SLA under all circumstances.

Primary Pathway: Gemini AI

The system uses the @google/generative-ai SDK configured to prompt gemini-1.5-flash with a maximum timeout of 8000ms. The system prompts Gemini with detailed guidelines and structured rules to return a clean, parseable JSON payload matching the curriculum requirements.

Secondary Pathway: Deterministic Templates

If Gemini throws an API error, times out, or has no key configured, the orchestrator instantly falls back to a template-driven engine. This engine contains 47 pre-authored lessons spanning all themes and age bands. The engine selects a template, randomizes elements like materials where applicable, and returns a verified lesson layout in under 10 milliseconds.

6. Quality Assurance & Test Strategy

The application maintains a high-coverage test suite across multiple testing methodologies:

Unit & Integration Tests (Vitest)

- Auth Endpoints: Validates registration, duplicate email rejection, short passwords, and valid login checks.
- Fallback Templates: Validates that all 47 templates have valid objectives, activities, stanzas, and non-empty materials lists.
- Gemini Client: Simulates successful Gemini JSON extraction and handles mock timeouts / error fallbacks.
- Lessons & Export: Tests fetching, pagination, delete cascades, and PDF binary streams.

Performance & Security Smoke Tests

- perf.smoke.test.ts: Measures generation latency, asserting that Gemini response averages under 10 seconds, and fallbacks process under 20 milliseconds.
- security.smoke.test.ts: Verifies middleware locks, path-traversal blocks, and prevents teachers from accessing or deleting other teachers' lessons.

End-to-End & Accessibility Tests (Playwright)

- E2E happy path: Tests registering, generating a lesson, viewing its details, and exporting it.
- WCAG 2.2 AA Audit: Uses @axe-core/playwright to scan the UI, ensuring proper labels, contrast, and structure for screen-reader compatibility.

7. Future Recommendations & Strategic Roadmap

To transform this application from a highly reliable MVP into an enterprise-grade curriculum suite, we propose the following strategic enhancements:

1. Intelligent Response Caching (Redis)

Preschool lessons generated with the same theme and age band do not need to hit the Gemini API repeatedly. We recommend placing a Redis cache in front of the generator service. If a user requests a lesson on "Seasons & Weather" for "Ages 3-4" and it was generated recently, we can return it from the cache. This will drastically reduce Gemini API usage costs, optimize bandwidth, and deliver sub-100ms response times.

2. Vector Database & RAG (Retrieval-Augmented Generation)

Instead of purely generating lessons from scratch, the system can leverage a vector database (e.g. pgvector in PostgreSQL) containing thousands of preschool curriculum guides, state standards, and teaching plans. By embedding the user's prompt and performing a similarity search, we can retrieve relevant curriculum context and pass it to Gemini. This guarantees that generated activities align with official early-childhood developmental benchmarks.

3. Multi-Modal Visual Asset Generator (Fal.ai)

Currently, the system generates "worksheet ideas" as text description. We suggest integrating Fal.ai or Stable Diffusion APIs on the backend. When a lesson is generated, teachers can click "Generate Printable Assets" to automatically produce ready-to-print PDF worksheets, coloring sheets, or visual flashcards based on the theme, which would double user engagement.

4. Mobile Application via React Native (Expo)

Preschool teachers are constantly on their feet and rarely use laptops in the classroom. Wrapping the existing React codebase or building a dedicated mobile app using React Native/Expo will allow teachers to view, share, and manage lesson plans on their phones or tablets. We can also integrate native notifications for daily classroom schedules.

5. Collaborative Teacher Spaces & Sharing

Enable schools and centers to register as organizations. Teachers can collaborate, share successful lesson designs, comment, rate activities, and build a unified curriculum calendar. This transforms the standalone tool into a collaborative B2B platform.